Transaction Lifecycle in Cosmos SDK: Complete Analysis

목차

- 개요
- Transaction 구조
- Transaction 생성
- Transaction 제출
- Mempool 처리
- Transaction 실행
- Gas 계산 메커니즘
- Event System
- 실습: Transaction Tracer 개발
- 참고 자료

개요

Transaction은 블록체인 상태를 변경하는 유일한 방법입니다. 이 문서에서는 트랜잭션이 생성되어 실행되고 최종적으로 블록에 포함되기까지의 전체 생명주기를 추적합니다.

학습 목표:

- Transaction 구조 완전 이해
- 생성부터 실행까지 전체 플로우 추적
- Gas 계산 메커니즘 상세 분석
- Transaction tracer 도구 개발

예상 학습 시간: 4-5시간

Transaction 구조

Tx 인터페이스

```go
// types/tx_msg.go
type Tx interface {
    // GetMsgs returns the list of messages in the transaction
    GetMsgs() []Msg

    // ValidateBasic performs stateless validation
    ValidateBasic() error
}
```

Standard Transaction

```
// tx/types/tx.pb.go
type Tx struct {
    // Body contains the messages and metadata
    Body *TxBody `protobuf:"bytes,1,opt,name=body,proto3" json:"body,omitempty"`

    // AuthInfo contains the fee and signer info
    AuthInfo *AuthInfo `protobuf:"bytes,2,opt,name=auth_info,json=authInfo,proto3" json:"auth_info,omitempty"`

    // Signatures from all signers
    Signatures [][]byte `protobuf:"bytes,3,rep,name=signatures,proto3" json:"signatures,omitempty"`
}
```

TxBody

```
type TxBody struct {
    // Messages to execute
    Messages []*types.Any `protobuf:"bytes,1,rep,name=messages,proto3" json:"messages,omitempty"`

    // Memo (optional)
    Memo string `protobuf:"bytes,2,opt,name=memo,proto3" json:"memo,omitempty"`

    // Timeout height (optional)
    TimeoutHeight uint64 `protobuf:"varint,3,opt,name=timeout_height,json=timeoutHeight,proto3" json:"timeout_height,omitempty"`

    // Extension options
    ExtensionOptions []*types.Any `protobuf:"bytes,1023,rep,name=extension_options,json=extensionOptions,proto3" json:"extension_options
}
```

AuthInfo

```
type AuthInfo struct {
    // Signer info (account, sequence, public key)
    SignerInfos []*SignerInfo `protobuf:"bytes,1,rep,name=signer_infos,json=signerInfos,proto3" json:"signer_infos,omitempty"`

    // Fee information
    Fee *Fee `protobuf:"bytes,2,opt,name=fee,proto3" json:"fee,omitempty"`

    // Tip (optional, for priority)
    Tip *Tip `protobuf:"bytes,3,opt,name=tip,proto3" json:"tip,omitempty"`
}

type SignerInfo struct {
    // Public key
    PublicKey *types.Any `protobuf:"bytes,1,opt,name=public_key,json=publicKey,proto3" json:"public_key,omitempty"`

    // Mode info (how to verify signature)
    ModeInfo *ModeInfo `protobuf:"bytes,2,opt,name=mode_info,json=modeInfo,proto3" json:"mode_info,omitempty"`

    // Sequence number (nonce)
    Sequence uint64 `protobuf:"varint,3,opt,name=sequence,proto3" json:"sequence,omitempty"`
}

type Fee struct {
    // Amount of tokens to pay
    Amount sdk.Coins `protobuf:"bytes,1,rep,name=amount,proto3,castrepeated=github.com/cosmos/cosmos-sdk/types.Coins" json:"amount"`

    // Gas limit
    GasLimit uint64 `protobuf:"varint,2,opt,name=gas_limit,json=gasLimit,proto3" json:"gas_limit,omitempty"`

    // Fee payer (optional)
    Payer string `protobuf:"bytes,3,opt,name=payer,proto3" json:"payer,omitempty"`

    // Fee granter (optional)
    Granter string `protobuf:"bytes,4,opt,name=granter,proto3" json:"granter,omitempty"`
}
```

Transaction 생성

1. Message 생성

```
// Example: Bank send message
msg := &banktypes.MsgSend{
    FromAddress: "cosmos1abc...",
    ToAddress:   "cosmos1xyz...",
    Amount:      sdk.NewCoins(sdk.NewInt64Coin("stake", 1000000)),
}
```

2. TxBuilder 사용

```
// client/tx/tx.go
txBuilder := clientCtx.TxConfig.NewTxBuilder()

// Set messages
err := txBuilder.SetMsgs(msg)

// Set gas limit
txBuilder.SetGasLimit(200000)

// Set fee
fee := sdk.NewCoins(sdk.NewInt64Coin("stake", 2000))
txBuilder.SetFeeAmount(fee)

// Set memo
txBuilder.SetMemo("My first transaction")

// Set timeout height (optional)
txBuilder.SetTimeoutHeight(12345)
```

3. Signing

```go
// Get signing data
signerData := authsigning.SignerData{
    ChainID:       "mychain-1",
    AccountNumber: 0,
    Sequence:      0,
}

// Sign transaction
signMode := signing.SignMode_SIGN_MODE_DIRECT
sigData := signing.SingleSignatureData{
    SignMode:  signMode,
    Signature: nil,
}

sig := signing.SignatureV2{
    PubKey:   privKey.PubKey(),
    Data:     &sigData,
    Sequence: signerData.Sequence,
}

err = txBuilder.SetSignatures(sig)

// Generate sign bytes
signBytes, err := clientCtx.TxConfig.SignModeHandler().GetSignBytes(
    signMode,
    signerData,
    txBuilder.GetTx(),
)

// Sign
sigBytes, err := privKey.Sign(signBytes)

// Update signature
sigData.Signature = sigBytes
sig.Data = &sigData
err = txBuilder.SetSignatures(sig)
```

4. Encoding

```go
// Encode to bytes
txBytes, err := clientCtx.TxConfig.TxEncoder()(txBuilder.GetTx())
```

Transaction 제출

Broadcasting Modes

1. Sync Mode (기본)

```go
// Broadcast and wait for CheckTx result
res, err := clientCtx.BroadcastTxSync(txBytes)

// Response includes:
// - TxHash
// - Code (0 = success)
// - Log
```

2. Async Mode

```
// Broadcast and return immediately
res, err := clientCtx.BroadcastTxAsync(txBytes)

// Only returns TxHash
```

3. Block Mode (레거시)

```
// Broadcast and wait for tx to be included in block
res, err := clientCtx.BroadcastTxCommit(txBytes)

// Response includes:
// - TxHash
// - CheckTx result
// - DeliverTx result
// - Height
```

Broadcasting 플로우

```
Client
  │
  │ 1. Encode Tx
  ▼
TxBytes
  │
  │ 2. HTTP/gRPC Request
  ▼
CometBFT RPC
  │
  │ 3. BroadcastTx
  ▼
Mempool
  │
  │ 4. CheckTx
  ▼
ABCI App
  │
  │ 5. Validate
  ▼
Add to Mempool
```

Mempool 처리

CheckTx 실행

```go
// baseapp/abci.go
func (app *BaseApp) CheckTx(req abci.RequestCheckTx) abci.ResponseCheckTx {
    var mode runTxMode

    switch req.Type {
    case abci.CheckTxType_New:
        mode = runTxModeCheck
    case abci.CheckTxType_Recheck:
        mode = runTxModeReCheck
    default:
        panic("unknown check tx type")
    }

    // Decode transaction
    tx, err := app.txDecoder(req.Tx)
    if err != nil {
        return sdkerrors.ResponseCheckTx(err, 0, 0, app.trace)
    }

    // Run tx in check mode
    gInfo, result, err := app.runTx(mode, req.Tx)
    if err != nil {
        return sdkerrors.ResponseCheckTx(err, gInfo.GasWanted, gInfo.GasUsed, app.trace)
    }

    return abci.ResponseCheckTx{
        Code:      0,
        GasWanted: int64(gInfo.GasWanted),
        GasUsed:   int64(gInfo.GasUsed),
        Events:    result.Events.ToABCIEvents(),
        Priority:  result.Priority,
    }
}
```

CheckTx vs DeliverTx

측면	CheckTx	DeliverTx
State	checkState (임시)	deliverState (영구)
목적	멤풀 검증	실제 실행
실행 횟수	여러 번 가능	블록당 1회
State 변경	커밋 안됨	커밋됨
Gas 소비	측정만	실제 차감

Transaction 실행

RunTx: 핵심 실행 로직

```go
// baseapp/baseapp.go
func (app *BaseApp) runTx(
    mode runTxMode,
    txBytes []byte,
) (gInfo sdk.GasInfo, result *sdk.Result, err error) {
    // ===== Phase 1: Decoding =====
    tx, err := app.txDecoder(txBytes)
    if err != nil {
        return sdk.GasInfo{}, nil, err
    }

    msgs := tx.GetMsgs()
    if err := validateBasicTxMsgs(msgs); err != nil {
        return sdk.GasInfo{}, nil, err
    }

    // ===== Phase 2: Context Setup =====
    var ctx sdk.Context
    switch mode {
    case runTxModeCheck:
        ctx = app.checkState.ctx.WithTxBytes(txBytes).WithIsCheckTx(true)
    case runTxModeSimulate:
        ctx = app.checkState.ctx.WithTxBytes(txBytes).WithIsCheckTx(true)
    case runTxModeDeliver:
        ctx = app.deliverState.ctx.WithTxBytes(txBytes).WithIsCheckTx(false)
    }

    // ===== Phase 3: Gas Setup =====
    gasWanted := uint64(0)

    var gasLimit uint64
    if feeTx, ok := tx.(sdk.FeeTx); ok {
        gasLimit = feeTx.GetGas()
    }

    if gasLimit == 0 {
        gasLimit = app.GetMaxGasLimit()
    }

    gasMeter := sdk.NewGasMeter(gasLimit)
    ctx = ctx.WithGasMeter(gasMeter)

    defer func() {
        // Capture gas info
        gInfo = sdk.GasInfo{
            GasWanted: gasWanted,
            GasUsed:   ctx.GasMeter().GasConsumed(),
        }

        // Handle panic (usually OutOfGas)
        if r := recover(); r != nil {
            switch rType := r.(type) {
            case sdk.ErrorOutOfGas:
                err = sdkerrors.Wrapf(sdkerrors.ErrOutOfGas,
                    "out of gas in location: %v; gasWanted: %d, gasUsed: %d",
                    rType.Descriptor, gasWanted, ctx.GasMeter().GasConsumed())
            default:
                panic(r)
            }
        }
    }()

    // ===== Phase 4: AnteHandler =====
    if app.anteHandler != nil {
        newCtx, err := app.anteHandler(ctx, tx, mode == runTxModeSimulate)
        if err != nil {
            return gInfo, nil, err
        }
        ctx = newCtx
    }
```

```go
    // ===== Phase 5: Message Execution =====
    msgResults := make([]*sdk.Result, len(msgs))

    for i, msg := range msgs {
        msgResult, err := app.handleMsg(ctx, msg, mode)
        if err != nil {
            return gInfo, nil, err
        }
        msgResults[i] = msgResult
    }

    // ===== Phase 6: PostHandler (optional) =====
    if app.postHandler != nil {
        newCtx, err := app.postHandler(ctx, tx, mode == runTxModeSimulate)
        if err != nil {
            return gInfo, nil, err
        }
        ctx = newCtx
    }

    // ===== Phase 7: Result Aggregation =====
    result = &sdk.Result{
        Data:   []byte{},
        Events: ctx.EventManager().ABCIEvents(),
        Log:    "",
    }

    return gInfo, result, nil
}
```

Message Handling

```go
func (app *BaseApp) handleMsg(
    ctx sdk.Context,
    msg sdk.Msg,
    mode runTxMode,
) (*sdk.Result, error) {
    // Route message to appropriate handler
    msgRoute := msg.Route()
    handler := app.router.Route(ctx, msgRoute)

    if handler == nil {
        return nil, sdkerrors.Wrapf(sdkerrors.ErrUnknownRequest,
            "unrecognized message route: %s", msgRoute)
    }

    // Execute handler
    msgResult, err := handler(ctx, msg)
    if err != nil {
        return nil, err
    }

    return msgResult, nil
}
```

Gas 계산 메커니즘

Gas 개념

Gas는 연산 비용을 측정하는 단위입니다.

```go
// types/gas.go
type Gas = uint64

type GasMeter interface {
    GasConsumed() Gas
    GasConsumedToLimit() Gas
    Limit() Gas
    ConsumeGas(amount Gas, descriptor string)
    RefundGas(amount Gas, descriptor string)
    IsPastLimit() bool
    IsOutOfGas() bool
}
```

Gas Meter 구현

```go
// types/gas.go
type basicGasMeter struct {
    limit    Gas
    consumed Gas
}

func (g *basicGasMeter) ConsumeGas(amount Gas, descriptor string) {
    var overflow bool
    g.consumed, overflow = addUint64Overflow(g.consumed, amount)

    if overflow {
        panic(ErrorGasOverflow{descriptor})
    }

    if g.consumed > g.limit {
        panic(ErrorOutOfGas{descriptor})
    }
}

func (g *basicGasMeter) RefundGas(amount Gas, descriptor string) {
    if g.consumed < amount {
        panic("negative gas consumed")
    }
    g.consumed -= amount
}
```

Gas 소비 지점

1. Transaction Size

```go
// ante/fee.go
txSize := len(ctx.TxBytes())
gasForTx := params.TxSizeCostPerByte * uint64(txSize)
ctx.GasMeter().ConsumeGas(gasForTx, "tx size")
```

2. Signature Verification

```go
// ante/sigverify.go
pubKey := account.GetPubKey()
sigCost := params.SigVerifyCostED25519
if _, ok := pubKey.(*secp256k1.PubKey); ok {
    sigCost = params.SigVerifyCostSecp256k1
}
ctx.GasMeter().ConsumeGas(sigCost, "signature verification")
```

3. Store Access

```go
// store/gaskv/store.go
func (gs *Store) Get(key []byte) []byte {
    gs.gasMeter.ConsumeGas(gs.gasConfig.ReadCostFlat, "read flat")

    value := gs.parent.Get(key)

    // Consume gas per byte
    gs.gasMeter.ConsumeGas(
        gs.gasConfig.ReadCostPerByte * Gas(len(key)),
        "read per byte (key)",
    )
    gs.gasMeter.ConsumeGas(
        gs.gasConfig.ReadCostPerByte * Gas(len(value)),
        "read per byte (value)",
    )

    return value
}

func (gs *Store) Set(key, value []byte) {
    gs.gasMeter.ConsumeGas(gs.gasConfig.WriteCostFlat, "write flat")
    gs.gasMeter.ConsumeGas(
        gs.gasConfig.WriteCostPerByte * Gas(len(key)),
        "write per byte (key)",
    )
    gs.gasMeter.ConsumeGas(
        gs.gasConfig.WriteCostPerByte * Gas(len(value)),
        "write per byte (value)",
    )

    gs.parent.Set(key, value)
}
```

4. Module Operations

```go
// x/bank/keeper/send.go
func (k Keeper) SendCoins(
    ctx sdk.Context,
    fromAddr sdk.AccAddress,
    toAddr sdk.AccAddress,
    amt sdk.Coins,
) error {
    // Consume gas for this operation
    ctx.GasMeter().ConsumeGas(k.config.SendCoinsGas, "send coins")

    // Perform operation
    err := k.subUnlockedCoins(ctx, fromAddr, amt)
    if err != nil {
        return err
    }

    err = k.addCoins(ctx, toAddr, amt)
    if err != nil {
        return err
    }

    return nil
}
```

Gas Configuration

```go
// types/gas.go
type GasConfig struct {
    ReadCostFlat     Gas
    ReadCostPerByte  Gas
    WriteCostFlat    Gas
    WriteCostPerByte Gas
    IterNextCostFlat Gas
}

var DefaultGasConfig = GasConfig{
    ReadCostFlat:     1000,
    ReadCostPerByte:  3,
    WriteCostFlat:    2000,
    WriteCostPerByte: 30,
    IterNextCostFlat: 30,
}
```

Gas Price & Fee

```go
// Fee = GasLimit * GasPrice

// Example
gasLimit := uint64(200000)
gasPrice := sdk.NewDecCoinFromDec("stake", sdk.NewDec(25))  // 0.000025 stake per gas

fee := gasPrice.Amount.MulInt64(int64(gasLimit))
// fee = 0.000025 * 200000 = 5 stake
```

Event System

Event 구조

```go
// types/events.go
type Event struct {
    Type       string
    Attributes []Attribute
}

type Attribute struct {
    Key   string
    Value string
}
```

Event 발생

```go
// x/bank/keeper/send.go
func (k Keeper) SendCoins(/* ... */) error {
    // ... send logic

    // Emit event
    ctx.EventManager().EmitEvents(sdk.Events{
        sdk.NewEvent(
            types.EventTypeTransfer,
            sdk.NewAttribute(types.AttributeKeyRecipient, toAddr.String()),
            sdk.NewAttribute(sdk.AttributeKeyAmount, amt.String()),
        ),
        sdk.NewEvent(
            sdk.EventTypeMessage,
            sdk.NewAttribute(sdk.AttributeKeyModule, types.AttributeValueCategory),
            sdk.NewAttribute(sdk.AttributeKeySender, fromAddr.String()),
        ),
    })

    return nil
}
```

Event Query

```bash
# Query by event
curl "localhost:26657/tx_search?query=\"transfer.recipient='cosmos1abc...'\"&prove=false"

# Query by height
curl "localhost:26657/block_results?height=12345"
```

실습: Transaction Tracer 개발

Tracer 구조

```go
// tracer/tracer.go
package tracer

import (
    sdk "github.com/cosmos/cosmos-sdk/types"
)

type TxTrace struct {
    TxHash        string
    Height        int64
    Timestamp     time.Time
    Sender        string
    Messages      []MsgTrace
    GasWanted     uint64
    GasUsed       uint64
    Fee           sdk.Coins
    Events        []sdk.Event
    Success       bool
    Error         string
}

type MsgTrace struct {
    Index     int
    Type      string
    Route     string
    GasUsed   uint64
    Events    []sdk.Event
    Success   bool
    Error     string
}

type Tracer struct {
    ctx     sdk.Context
    txTrace *TxTrace
}

func NewTracer(ctx sdk.Context, tx sdk.Tx) *Tracer {
    return &Tracer{
        ctx: ctx,
        txTrace: &TxTrace{
            TxHash:    fmt.Sprintf("%X", tmhash.Sum(ctx.TxBytes())),
            Height:    ctx.BlockHeight(),
            Timestamp: ctx.BlockTime(),
            Messages:  make([]MsgTrace, 0),
        },
    }
}

func (t *Tracer) TraceMsg(index int, msg sdk.Msg) {
    msgTrace := MsgTrace{
        Index: index,
        Type:  sdk.MsgTypeURL(msg),
        Route: msg.Route(),
    }

    t.txTrace.Messages = append(t.txTrace.Messages, msgTrace)
}

func (t *Tracer) SetGasInfo(gasWanted, gasUsed uint64) {
    t.txTrace.GasWanted = gasWanted
    t.txTrace.GasUsed = gasUsed
}

func (t *Tracer) SetFee(fee sdk.Coins) {
    t.txTrace.Fee = fee
}

func (t *Tracer) SetEvents(events sdk.Events) {
    t.txTrace.Events = events
}
```

```go
func (t *Tracer) SetResult(success bool, err error) {
    t.txTrace.Success = success
    if err != nil {
        t.txTrace.Error = err.Error()
    }
}

func (t *Tracer) GetTrace() *TxTrace {
    return t.txTrace
}

func (t *Tracer) PrintTrace() {
    trace := t.GetTrace()

    fmt.Println("╔═══════════════════════════════════════╗")
    fmt.Printf("║ Transaction Trace                       ║\n")
    fmt.Println("╟═══════════════════════════════════════╢")
    fmt.Printf("║ Hash:    %s\n", trace.TxHash[:16]+"...")
    fmt.Printf("║ Height:  %d\n", trace.Height)
    fmt.Printf("║ Success: %t\n", trace.Success)
    fmt.Println("╟═══════════════════════════════════════╢")
    fmt.Printf("║ Gas Wanted: %d\n", trace.GasWanted)
    fmt.Printf("║ Gas Used:   %d\n", trace.GasUsed)
    fmt.Printf("║ Gas Ratio:  %.2f%%\n", float64(trace.GasUsed)/float64(trace.GasWanted)*100)
    fmt.Println("╟═══════════════════════════════════════╢")
    fmt.Printf("║ Fee: %s\n", trace.Fee.String())
    fmt.Println("╟═══════════════════════════════════════╢")

    for i, msg := range trace.Messages {
        fmt.Printf("║ Message #%d\n", i)
        fmt.Printf("║   Type: %s\n", msg.Type)
        fmt.Printf("║   Gas:  %d\n", msg.GasUsed)
        fmt.Printf("║   Success: %t\n", msg.Success)
    }

    fmt.Println("╟═══════════════════════════════════════╢")
    fmt.Printf("║ Events: %d\n", len(trace.Events))
    for _, event := range trace.Events {
        fmt.Printf("║   - %s\n", event.Type)
    }
    fmt.Println("╚═══════════════════════════════════════╝")
}
```

통합

```go
// app/app.go
func (app *MyApp) DeliverTx(req abci.RequestDeliverTx) abci.ResponseDeliverTx {
    tx, err := app.txDecoder(req.Tx)
    if err != nil {
        return sdkerrors.ResponseDeliverTx(err, 0, 0, app.trace)
    }

    // Create tracer
    tracer := tracer.NewTracer(app.deliverState.ctx, tx)

    // Extract fee
    if feeTx, ok := tx.(sdk.FeeTx); ok {
        tracer.SetFee(feeTx.GetFee())
    }

    // Run tx
    gInfo, result, err := app.runTx(runTxModeDeliver, req.Tx)

    // Record gas info
    tracer.SetGasInfo(gInfo.GasWanted, gInfo.GasUsed)

    if result != nil {
        tracer.SetEvents(sdk.Events(result.Events))
        tracer.SetResult(true, nil)
    } else {
        tracer.SetResult(false, err)
    }

    // Print trace
    if app.traceEnabled {
        tracer.PrintTrace()
    }

    // Return response
    if err != nil {
        return sdkerrors.ResponseDeliverTx(err, gInfo.GasWanted, gInfo.GasUsed, app.trace)
    }

    return abci.ResponseDeliverTx{
        Code:      0,
        Data:      result.Data,
        Log:       result.Log,
        GasWanted: int64(gInfo.GasWanted),
        GasUsed:   int64(gInfo.GasUsed),
        Events:    result.Events,
    }
}
```

참고 자료

공식 문서

- Cosmos SDK Transactions
- Gas and Fees
- Events

소스 코드

- BaseApp RunTx
- AnteHandler
- Gas Meter

작성자: B-Harvest Core Developer Candidate **버전**: 1.0 **최종 수정일**: 2025-01-XX